



MSPR

Mise en œuvre d'un processus ETL
répondant au besoin d'une solution I.A

FORMATION : Développeur en Intelligence Artificielle

CERTIFICATION : RNCP36581, Bloc E6.1

ÉTABLISSEMENT : EPSI Lille

Membres du groupe

BILÉ JOHAN

DIA NABIL

HACCANDY JOSEPH

KUTUMBAKANA GLODY

Date de soutenance

28 avril 2026

TABLE DES MATIÈRES

Liste des acronymes	v
Introduction	1
Présentation du projet	2
Livrables	3
1 Les Données	5
1.1 Sélection et justification des données et de la base de données	5
1.1.1 Contexte de la collecte	5
1.1.2 Sources de données évaluées	5
1.1.3 Sources retenues	6
1.1.4 Sources écartées et justification	8
1.1.5 Justification du choix du SGBD : pourquoi PostgreSQL ?	8
1.2 Conformité RGPD	8
1.2.1 Nature des données traitées	9
1.2.2 Application des principes RGPD	9
1.3 Modèle Conceptuel de Données (MCD)	10
1.3.1 Schéma MCD	10
1.3.2 Logique générale du modèle	10
1.3.3 Description de chaque entité	11
1.3.4 Relations N-N : les tables exploite et utilisation	13
1.4 Intégration ETL via Talend	13
1.4.1 Job_Pays : Table cible : pays (Job 1/9)	13
1.4.2 Job_Gare : Table cible : gare (Job 2/9)	14
1.4.3 Job_Operateur : Table cible : operateur (Job 3/9)	15
1.4.4 Job_TypeTrain : Table cible : type_train (Job 4/9)	16
1.4.5 Job_Ligne : Table cible : ligne (Job 5/9)	16
1.4.6 Job_Trajet : Table cible : trajet (Job 6/9)	17
1.4.7 Job_Exploite : Table cible : exploite (Job 7/9)	18
1.4.8 Job_Itineraire : Table cible : itineraire (Job 8/9)	18
1.4.9 Job_Emission : Table cible : emission (Job 9/9)	19
1.5 Automatisation de l'intégration ETL	20
1.5.1 Contexte et choix technique	20
1.5.2 Structure du script de lancement	20

1.5.3	Équivalent sur Linux	22
2	L'API REST	23
2.1	Objectif du backend	23
2.2	Architecture technique	23
2.2.1	Structure des dossiers	23
2.2.2	Rôle de chaque couche	24
2.3	Prérequis et lancement	24
2.3.1	Dépendances Python	24
2.3.2	Configuration et lancement local	24
2.3.3	URLs utiles	24
2.4	Configuration FastAPI	25
2.5	Connexion base de données	25
2.6	Modèles de données principaux (ORM)	25
2.7	Schémas de réponse (Pydantic)	25
2.8	Documentation complète des endpoints	26
2.9	Gestion des erreurs et codes HTTP	26
2.10	Contrat backend ↔ dashboard	26
2.11	Vérification rapide (<i>smoke tests</i>)	27
2.12	Évolutions recommandées	27
3	Le Dashboard	28
3.1	Fonctionnalités principales du dashboard	28
3.1.1	Aperçu global	28
3.1.2	Cartographie du réseau	28
3.1.3	Impact environnemental	29
3.1.4	Qualité des données	29
3.2	Technologies utilisées et justification des choix	30
3.3	Apports du dashboard au projet OBRAIL EUROPE	31
	Conclusion	32
	Bibliographie et webographie	34

TABLE DES FIGURES

1	Architecture globale du projet OBRail EUROPE	3
1.1	Modèle Conceptuel de Données (MCD) du projet OBRail EUROPE	10
1.2	Schéma Talend : Job_Pays	13
1.3	Schéma Talend : Job_Gare	14
1.4	Schéma Talend : Job_Operateur	15
1.5	Schéma Talend : Job_TypeTrain	16
1.6	Schéma Talend : Job_Ligne	17
1.7	Schéma Talend : Job_Trajet	17
1.8	Schéma Talend : Job_Exploite	18
1.9	Schéma Talend : Job_Itineraire	19
1.10	Schéma Talend : Job_Emission	20
1.11	Processus d'automatisation du pipeline ETL	21
3.1	Dashboard : Aperçu global des indicateurs clés	28
3.2	Dashboard : Cartographie interactive du réseau ferroviaire européen	29
3.3	Dashboard : Comparaison des émissions CO ₂ train vs avion	29
3.4	Dashboard : Diagnostic de qualité des données	30

LISTE DES TABLEAUX

1.1	Inventaire des sources de données évaluées	6
1.2	Sources écartées et raisons de l'exclusion	8
1.3	Analyse des données au regard du RGPD	9
1.4	Application des principes du RGPD au projet	9
2.1	Rôle de chaque module du backend	24
2.2	Endpoints de l'API REST OBRail EUROPE	26
2.3	Codes HTTP utilisés	26
3.1	Technologies du dashboard et justification	30

LISTE DES ACRONYMES

Acronyme	Signification
ACID	Atomicité, Cohérence, Isolation, Durabilité
ADEME	Agence de l'Environnement et de la Maîtrise de l'Énergie
API	Application Programming Interface
BEIS	Department for Business, Energy & Industrial Strategy (UK)
CRUD	Create, Read, Update, Delete
ETL	Extract, Transform, Load
GTFS	General Transit Feed Specification
KPI	Key Performance Indicator
MCD	Modèle Conceptuel de Données
ORM	Object-Relational Mapping
REST	Representational State Transfer
RGPD	Règlement Général sur la Protection des Données
SGBD	Système de Gestion de Base de Données
UIC	Union Internationale des Chemins de fer

INTRODUCTION

Problématique

Comment collecter, organiser et rendre exploitables des données ferroviaires issues de dizaines de sources différentes, pour permettre à OBRail EUROPE de comparer objectivement l'impact environnemental du train et de l'avion sur les liaisons intra-européennes ?

C'est le point de départ de ce projet. OBRail EUROPE est un observatoire indépendant dont la mission est d'analyser les dessertes ferroviaires en Europe et de promouvoir le train comme alternative écologique à l'avion. Pour alimenter ses analyses, ses partenaires institutionnels et les futurs modèles d'intelligence artificielle, l'organisation a besoin d'une base de données fiable, structurée et automatiquement mise à jour.

Le problème, c'est que les données ferroviaires européennes sont éparpillées entre des dizaines d'acteurs différents (opérateurs nationaux, organismes statistiques, ONG, plateformes *open data*) et publiées dans des formats incompatibles entre eux : CSV, JSON, GTFS, Excel, etc. Sans travail de collecte, de nettoyage et d'organisation, ces données sont inexploitables.

Notre réponse à cette problématique est un pipeline ETL (*Extract, Transform, Load*) complet : un processus automatisé qui extrait les données de leurs sources, les transforme et les nettoie, puis les charge dans une base de données PostgreSQL structurée. Ce rapport technique documente la totalité de ce travail.

Plan du document. Ce document est organisé en trois grandes parties, chacune correspondant à un axe technique du projet :

- **Partie 1, Les données :** sélection des sources, construction du modèle de données et alimentation de la base via TALEND OPEN STUDIO.
- **Partie 2, L'API REST :** exposition des données via une interface web standardisée développée avec FastAPI.
- **Partie 3, Le Dashboard :** visualisation interactive des données à travers un tableau de bord développé avec Streamlit et Plotly.

PRÉSENTATION DU PROJET

OBRAIL EUROPE est un observatoire indépendant créé en 2018, spécialisé dans l'analyse des flux ferroviaires européens et la promotion de la mobilité durable. Son objectif est de mesurer concrètement le rôle des trains de jour et des trains de nuit dans la construction d'un réseau de transport bas-carbone, et d'évaluer leur potentiel comme alternative crédible à l'avion sur les liaisons intra-européennes.

Pour mener à bien cette mission, OBRAIL EUROPE a besoin d'un référentiel de données solide. C'est dans ce cadre que notre équipe a été mandatée pour concevoir et mettre en œuvre un processus ETL complet, depuis la collecte des données brutes jusqu'à leur mise à disposition via une base de données relationnelle.

Contexte et enjeux

Le projet s'inscrit dans deux grandes stratégies européennes :

- Le **Green Deal européen**, qui vise la neutralité carbone à l'horizon 2050 et pour lequel le développement du rail représente un levier majeur.
- Le programme **TEN-T** (*Trans-European Transport Network*), qui cherche à renforcer l'intégration et la compétitivité du réseau ferroviaire européen.

Dans ce contexte, OBRAIL EUROPE doit répondre à trois contraintes majeures identifiées dans son étude préliminaire. Premièrement, les données ferroviaires sont dispersées entre de nombreux acteurs utilisant des formats incompatibles. Deuxièmement, la qualité des jeux de données est hétérogène : doublons, valeurs manquantes, formats incohérents. Troisièmement, il n'existe pas de référentiel commun permettant de comparer facilement les dessertes ferroviaires entre pays européens.

Objectifs du projet

L'objectif principal est de mettre en place un entrepôt de données unifié et automatisé permettant de collecter, transformer, stocker et exposer des données fiables sur les dessertes ferroviaires européennes. Cet entrepôt doit devenir une source de référence pour les analyses d'OBRAIL EUROPE, pour ses partenaires institutionnels (Commission européenne, Parlement européen) et pour les ONG environnementales comme Back-on-Track.

Concrètement, cela signifie : agréger des données issues de multiples opérateurs et sources

open data, garantir leur cohérence et leur traçabilité, permettre la comparaison entre trains de jour et trains de nuit, rendre les données accessibles via une API REST, et préparer le terrain pour les futures étapes du projet (modèles prédictifs, mise en production).

Architecture générale

Avant de rentrer dans le détail technique, il est important de comprendre comment les trois grandes parties du projet s'articulent entre elles. Le schéma ci-dessous représente l'architecture globale du système.

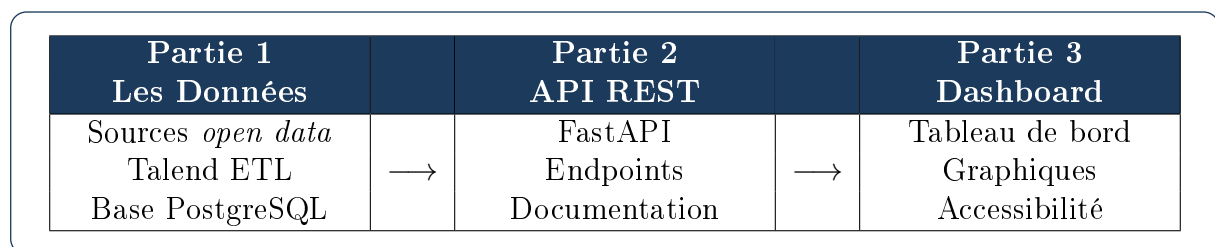


Figure 1: Architecture globale du projet OBRail EUROPE

La Partie 1 constitue le socle de tout le projet. Sans données propres, structurées et fiables, ni l'API ni la visualisation ne peuvent fonctionner. **La Partie 2** expose ces données via une API REST standardisée. **La Partie 3** propose enfin une interface de visualisation accessible à tout public.

Livrables

L'ensemble du code source, des fichiers de configuration et des ressources du projet est versionné et accessible publiquement sur GitHub :

Dépôt GitHub du projet

<https://github.com/JXPM/MSPR/>

Le dépôt contient les livrables suivants :

- **Pipeline ETL** : les 9 jobs Talend exportés, le script d'automatisation `lancement.bat` et les fichiers de données sources.
- **Backend API REST** : le code FastAPI complet (routes, services, modèles ORM, schémas Pydantic) et le fichier `requirements.txt`.
- **Dashboard** : l'application Streamlit avec ses composants de visualisation, la configuration Plotly/Mapbox et le service d'appel API.

- **Base de données** : les scripts SQL de création des tables, les contraintes d'intégrité et un dump de référence.
- **Documentation** : le présent rapport technique au format LaTeX.

CHAP 1 : LES DONNÉES

Sélection des sources • Modèle de données • ETL Talend

1.1 Sélection et justification des données et de la base de données

1.1.1 Contexte de la collecte

Les données ferroviaires européennes sont publiées par une multitude d'acteurs différents : opérateurs nationaux (SNCF, ÖBB, DB...), gestionnaires d'infrastructure, institutions statistiques européennes comme Eurostat, et ONG comme Back-on-Track. Chaque source utilise son propre format (CSV, JSON, flux GTFS, fichiers Excel, TSV), ce qui rend leur intégration complexe.

Pour faire les bons choix, nous avons appliqué quatre critères de sélection :

1. La **pertinence** de la source pour au moins une de nos tables.
2. La **fiabilité** de l'organisme qui la publie.
3. La **disponibilité** sous licence *open data*.
4. La **complétude** suffisante des données pour être utilisables sans traitement disproportionné.

1.1.2 Sources de données évaluées

Nous avons recensé et évalué l'ensemble des sources disponibles avant de décider lesquelles intégrer. Le tableau 1.1 présente cette analyse complète.

Table 1.1: Inventaire des sources de données évaluées

Fichier / Source	Origine	Format	Retenu ?
stations.csv	transport.data.gouv.fr	.csv	Oui
02_bot_agencies.json	Back-on-Track (GitHub)	.json	Oui
02_bot_routes.json	Back-on-Track (GitHub)	.json	Oui
02_bot_trips.json	Back-on-Track (GitHub)	.json	Oui
02_bot_trip_stop.json	Back-on-Track (GitHub)	.json	Oui
iso-3166.csv	ISO.org	.csv	Oui
bilans-des-emissions-sncf.csv	SNCF Open Data	.csv	Non
emission-co2-perimetre-*.csv	data.gouv.fr (SNCF)	.csv	Non
estat_rail_tf_traveh.tsv	Eurostat	.tsv	Non
stiptheid-van-eurostar.json	Infrabel Open Data	.json	Non
station_to_station.csv	Infrabel Open Data	.csv	Non
Export SNCF GTFS	SNCF Open Data	GTFS	Non
eu-airports.csv	OurAirports	.csv	Non
base-carboner-*.xlsx/.csv	Base Carbone (ADEME)	.xlsx/.csv	Non

1.1.3 Sources retenues

stations.csv : transport.data.gouv.fr

Tables alimentées : Gare, Pays

C'est la source la plus complète et la plus fiable disponible en *open data* pour les gares ferroviaires européennes. Ce fichier, publié par la plateforme officielle des données de transport du gouvernement français, contient plus de 70 000 gares avec leurs coordonnées GPS, leurs codes UIC officiels et leurs codes pays. Sans ce fichier, il serait impossible de géolocaliser les arrêts de nos trajets ou d'identifier les pays du réseau.

02_bot_*.json : Back-on-Track.eu

Tables alimentées : Operateur, Ligne, Trajet, Type_train, Itinéraire, Emission

Back-on-Track est l'ONG européenne de référence pour les trains de nuit. Elle maintient une base de données structurée et régulièrement mise à jour, directement alignée avec les besoins d'OBRAIL EUROPE. Ces quatre fichiers JSON couvrent la quasi-totalité des tables métier de notre modèle. Le fichier `trips.json` est utilisé en double rôle : source des trajets ferroviaires (`Job_Trajet`) et source des émissions CO₂ via les champs `emissions_co2e` et `distance` (`Job_Emission`). L'empreinte carbone de l'avion est calculée dans Talend avec le facteur officiel ADEME/BEIS de 0,158 kg de CO₂ par passager par kilomètre.

iso-3166.csv : ISO.org

Tables alimentées : Pays (enrichissement)

Ce fichier de référence internationale fournit les codes officiels de tous les pays (codes alpha-2 comme FR, DE, IT) et leurs noms complets. Il est utilisé comme table de correspondance pour normaliser les codes pays issus de `stations.csv`. Sans ce référentiel, les codes pays seraient des chaînes de caractères non vérifiées, sources d'incohérences dans toute la base.

1.1.4 Sources écartées et justification

Table 1.2: Sources écartées et raisons de l'exclusion

Source écartée	Raison de l'exclusion
bilans-des-emissions-sncf.csv	Données agrégées au niveau de l'entreprise SNCF entière (bilan annuel global). Impossible de les relier à un trajet précis.
emission-co2-perimetre-*.csv	Couverture limitée aux liaisons domestiques françaises. Seulement 3 paires de villes communes sur 119 avec nos données Back-on-Track.
base-carboner-*.xlsx/.csv	Facteurs d'émission génériques. Nous utilisons directement ces facteurs dans nos calculs Talend plutôt que d'intégrer les fichiers.
estat_rail_tf_traveh.tsv	Statistiques agrégées par pays et par année. Trop macroscopiques pour nos tables qui raisonnent au niveau du trajet ou de la ligne.
Export SNCF GTFS	Format GTFS couvrant uniquement les liaisons domestiques françaises. Hors périmètre européen international.
stiptheid-van-eurostar.json	Données de ponctualité Eurostar uniquement. Hors périmètre de notre modèle actuel.
station_to_station.csv	Distances entre gares belges uniquement. Couverture géographique trop limitée.
eu-airports.csv	Liste d'aéroports. Non nécessaire pour un modèle centré sur le ferroviaire.

1.1.5 Justification du choix du SGBD : pourquoi PostgreSQL ?

Pourquoi pas le Big Data ? Le Big Data se justifie quand on traite des volumes de données énormes (des téraoctets), en flux continu, et sous des formes non structurées. Notre projet n'entre dans aucune de ces catégories : quelques dizaines de milliers d'enregistrements, des données statiques chargées par batch, parfaitement structurées. Une architecture Hadoop ou une base NoSQL serait largement surdimensionnée.

Pourquoi PostgreSQL ? Les données ferroviaires sont par nature relationnelles : une gare appartient à un pays, un trajet appartient à une ligne, une émission est générée par un trajet. PostgreSQL s'est imposé naturellement pour quatre raisons :

1. C'est un outil **gratuit et open source**.
2. Il est **recommandé dans le cahier des charges** d'OBRAIL EUROPE.
3. Il est livré avec **pgAdmin**, une interface graphique complète pour gérer la base.
4. Il garantit la conformité ACID, ce qui assure l'**intégrité des données** lors des charge-ments ETL successifs.

1.2 Conformité RGPD

1.2.1 Nature des données traitées

Avant d'intégrer quoi que ce soit, nous avons analysé la nature de toutes les données du projet au regard du RGPD (*Règlement Général sur la Protection des Données*). Le RGPD protège les données à caractère personnel, c'est-à-dire toute information permettant d'identifier directement ou indirectement une personne physique.

Table 1.3: Analyse des données au regard du RGPD

Catégorie	Exemples concrets	Personnelles ?
Géographiques	Coordonnées GPS de gares, codes UIC	Non
Opérationnelles	Horaires de trains, noms de lignes	Non
Environnementales	Émissions CO ₂ par trajet en kg	Non
De référence	Codes pays ISO, noms d'opérateurs	Non
De traçabilité	URLs des sources, noms de fichiers	Non

Notre base de données ne contient **aucune donnée personnelle**. Elle ne stocke ni noms, ni prénoms, ni adresses e-mail, ni aucune information permettant d'identifier une personne. Le projet se situe donc en dehors du champ d'application direct du RGPD. Toutefois, par souci de rigueur et de transparence, OBRail EUROPE applique tout de même les principes du RGPD dans la gestion des données.

1.2.2 Application des principes RGPD

Table 1.4: Application des principes du RGPD au projet

Principe	Application concrète dans le projet
Transparence et traçabilité	Chaque source de données intégrée est documentée avec son URL d'origine, son nom et son format. N'importe qui peut retrouver d'où vient chaque information.
Minimisation	Nous n'intégrons que les champs strictement nécessaires. Par exemple, <code>stations.csv</code> contient des dizaines de colonnes : nous n'en conservons que cinq (code UIC, nom, latitude, longitude, pays).
Exactitude et qualité	Le pipeline ETL intègre des contrôles de qualité à chaque étape : filtrage des valeurs nulles, suppression des doublons, normalisation des formats.
Sécurité	La base PostgreSQL est hébergée en environnement local, protégée par authentification. L'API REST expose uniquement des données consolidées et validées, sans accès direct à la base.
Documentation	L'ensemble des choix techniques et des transformations appliquées est documenté dans ce rapport, assurant la reproductibilité du projet.

1.3 Modèle Conceptuel de Données (MCD)

Le MCD (*Modèle Conceptuel de Données*) représente la structure logique de notre base de données, indépendamment de toute considération technique. C'est la carte du territoire : il montre quelles informations nous stockons et comment elles sont reliées entre elles.

1.3.1 Schéma MCD

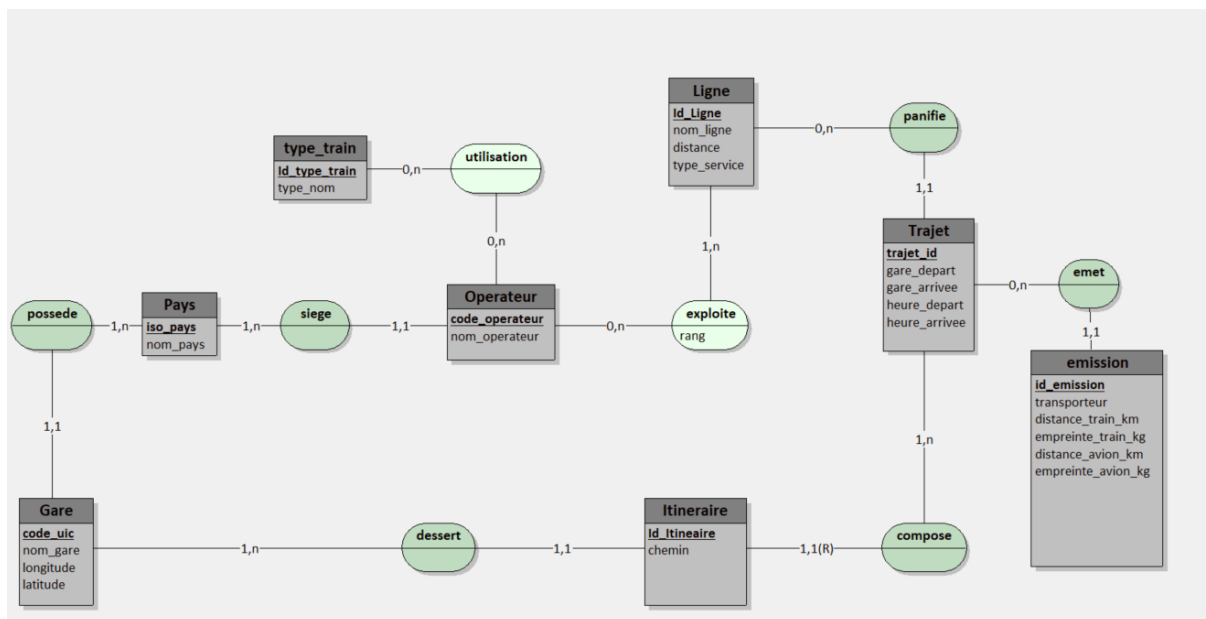


Figure 1.1: Modèle Conceptuel de Données (MCD) du projet OBRail EUROPE

1.3.2 Logique générale du modèle

Notre base de données est organisée autour de deux axes principaux qui se rejoignent :

Axe géographique. Les **pays** contiennent des **gares**. Chaque gare appartient à un pays, ce qui permet de situer les différents points du réseau ferroviaire.

Axe opérationnel. Les **opérateurs** exploitent des **lignes** ferroviaires. Sur ces lignes circulent des **trajets**, qui représentent les déplacements des trains entre une gare de départ et une gare d'arrivée. Chaque trajet est composé de plusieurs **arrêts**, décrits dans la table **Itinéraire**, qui indique le chemin suivi par le train entre les gares.

Enfin, la table **Emission** ajoute une dimension environnementale au modèle. Elle permet d'associer à chaque trajet les émissions de CO₂ du train et celles de l'avion équivalent, afin de comparer leur impact environnemental.

1.3.3 Description de chaque entité

Pays

Clé primaire	<code>iso_pays</code> (VARCHAR 2) : code ISO 3166 officiel
Rôle	Table de référence géographique. Point d’ancrage pour les tables Gare et Opérateur .
Relations	Un pays héberge plusieurs gares (1,n). Un pays est le siège d’un ou plusieurs opérateurs (1,n).
Justification	La normalisation des codes pays évite les saisies incohérentes (France, FR, FRA...).

Gare

Clé primaire	<code>code_uic</code> (VARCHAR 10) : identifiant UIC officiel.
Rôle	Table centrale géographique. Localisation précise (latitude, longitude) et rattachement à un pays.
Relations	Appartient à un pays (FK). Peut être un arrêt de plusieurs itinéraires (1,n).
Justification	Le code UIC est le standard international, garantissant l’unicité et la compatibilité avec d’autres bases ferroviaires.

Opérateur

Clé primaire	<code>code_operateur</code> (VARCHAR 100) : identifiant issu de <code>agency_id</code> (Back-on-Track).
Rôle	Référence toutes les compagnies ferroviaires (SNCF, ÖBB, DB, Trenitalia...).
Relations	Exploite plusieurs lignes via exploite (N-N). Utilise plusieurs types de trains via utilisation (N-N).
Justification	Permet l’analyse par opérateur : couverture territoriale, types de trains utilisés.

Type de train (Type_train)

Clé primaire	<code>id_type_train</code> (SERIAL) et <code>type_nom</code> (VARCHAR 50).
Rôle	Référentiel des catégories de matériel roulant : IR, EC, EN, ICE, TGV, Intercity...
Relations	Utilisé par un ou plusieurs opérateurs via utilisation (N-N).
Justification	La distinction jour/nuit est au cœur de la mission d’OBRail EUROPE.

Ligne

Clé primaire	<code>id_ligne</code> (INT) : conserve le <code>route_id</code> original pour garantir la cohérence des jointures.
Rôle	Représente une liaison ferroviaire avec sa distance et son type de service : JOUR ou NUIT.
Relations	Composée de plusieurs trajets (1,n). Exploitée par un ou plusieurs opérateurs via <code>exploite</code> (N-N).
Justification	La contrainte <code>CHECK</code> sur <code>type_service</code> garantit le classement JOUR/-NUIT.

Trajet

Clé primaire	<code>trajet_id</code> (VARCHAR 50) : correspond au <code>trip_id</code> des fichiers sources.
Rôle	Instance concrète d'une ligne : circulation réelle avec gares de départ/arrivée et horaires.
Relations	Appartient à une ligne (FK). Composée de plusieurs arrêts (1,n vers <code>Itinéraire</code>). Génère une émission CO ₂ (0,1 vers <code>Emission</code>).
Justification	Séparer <code>Ligne</code> et <code>Trajet</code> reflète la réalité ferroviaire et suit le standard GTFS.

Itinéraire

Clé primaire	Clé composite : (<code>id_itineraire</code> , <code>trajet_id</code>).
Rôle	Table de détail : chaque arrêt intermédiaire avec son numéro d'ordre et le code UIC de la gare.
Relations	Appartient à un trajet (FK). Correspond à une gare précise (FK vers <code>Gare</code>).
Justification	Indispensable pour répondre à : Quelles gares sont desservies sur Paris-Vienne ?

Emission

Clé primaire	<code>id_emission</code> (SERIAL auto-incrémenté).
Rôle	Table environnementale centrale. Stocke l'empreinte CO ₂ du train (issue de <code>trips.json</code>) et l'empreinte CO ₂ de l'avion équivalent (calculée : distance × 0,158 kg CO ₂ /km).
Relations	Générée par un trajet précis (FK vers <code>Trajet</code> , relation 1,1).
Justification	Cœur de la mission d'OBRAIL EUROPE : démontrer que le train est plus écologique. Sur 410 trajets, 310 disposent de données CO ₂ .

1.3.4 Relations N-N : les tables exploite et utilisation

Deux associations sont de type plusieurs-à-plusieurs (N-N). La table **exploite** relie **Opérateur** et **Ligne** avec un attribut **rang** qui précise si l'opérateur est le principal ou un co-exploitant : information utile sur les lignes internationales comme Paris–Vienne où SNCF et ÖBB co-exploitent le service. La table **utilisation** relie **Opérateur** et **Type_train**.

1.4 Intégration ETL via Talend

Cette section détaille les 9 jobs Talend développés pour alimenter la base de données. Chaque job correspond à une table cible et s'exécute dans un ordre précis qui respecte les dépendances entre tables.

Ordre d'exécution du pipeline

Pays (1) → Gare (2) → Opérateur (3) → TypeTrain (4) → Ligne (5) → Trajet (6) → Exploite (7) → Itinéraire (8) → Emission (9)

Comment lire les schémas Talend ?

Dans Talend, chaque composant est une boîte connectée par des flèches. Une **flèche orange** (*row Main*) = flux principal de données. Une **flèche verte** (*row Uniques*, *row Filtre*) = données filtrées ou déduplicuées. Une **flèche pointillée orange** = données chargées en mémoire pour une jointure (*lookup*). Les chiffres en vert indiquent combien de lignes ont traversé chaque connexion lors de l'exécution.

1.4.1 Job_Pays : Table cible : pays (Job 1/9)

Objectif

Extraire les pays uniques depuis **stations.csv**, les enrichir avec les codes ISO officiels depuis **iso-3166.csv**, éliminer les doublons et insérer le résultat dans la table **pays**. C'est le premier job car la table **pays** est une table racine.

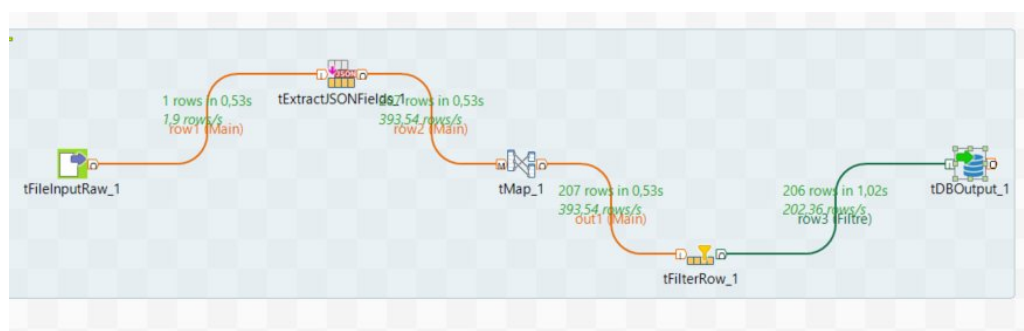


Figure 1.2: Schéma Talend : Job_Pays

tFileInputDelimited_1 : `stations.csv`. Lecture du fichier avec séparateur point-virgule, encodage UTF-8. Seule la colonne `country` est utilisée.

tUniqRow : **Suppression des doublons**. Clé de déduplication sur `country`. Résultat : 43 pays uniques conservés sur 70 600 lignes lues.

tFileInputDelimited_2 : `iso-3166.csv`. Dictionnaire officiel faisant correspondre le nom du pays à son code alpha-2 normalisé. Branché en *lookup* dans le tMap.

tMap : **Jointure et mapping final**. Jointure *Inner Join* entre les deux sources. Colonnes de sortie : `iso_pays` (code alpha-2) et `nom_pays` (nom officiel).

tDBOutput : **Écriture en base**. Insertion dans la table `pays` avec les colonnes `iso_pays` (clé primaire) et `nom_pays`.

1.4.2 Job_Gare : Table cible : gare (Job 2/9)

Objectif

Extraire les informations détaillées de chaque gare depuis `stations.csv` : code UIC, nom, coordonnées GPS et pays : et les insérer dans la table `gare` après nettoyage et déduplication. Avec plus de 23 770 gares insérées, c'est le job qui produit le plus grand volume de données.

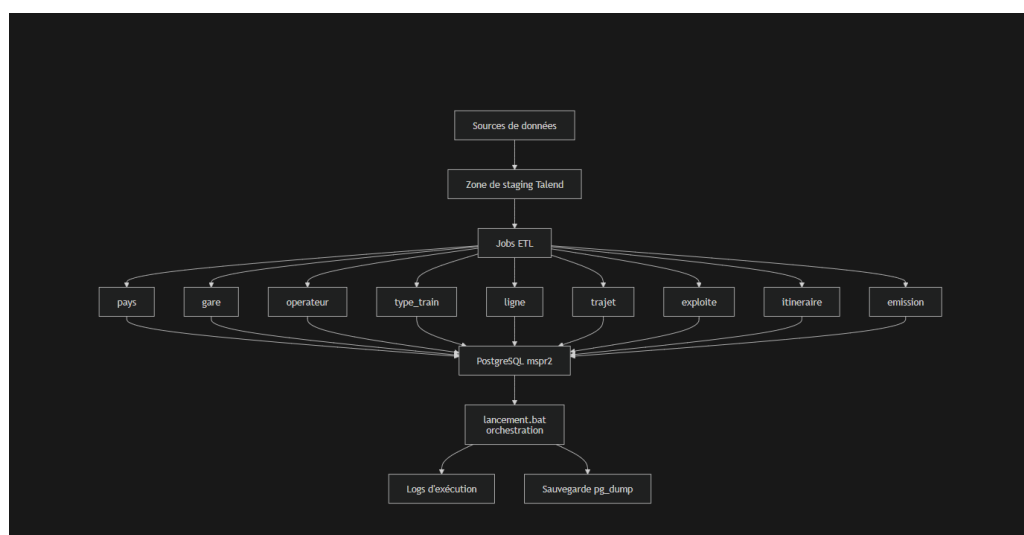


Figure 1.3: Schéma Talend : Job_Gare

tFileInputDelimited_1. Colonnes exploitées : `uic`, `name`, `latitude`, `longitude`, `country`.

tFilterRow. Élimination des lignes avec un code UIC vide (nécessaire car c'est la clé primaire).

tUniqRow. Clé de déduplication sur `uic`. Résultat : 23 770 gares uniques sur 70 600 lignes en entrée.

tMap. Renommage des colonnes : `uic` → `code_uic`, `name` → `nom_gare`, `country` → `iso_pays`.

tDBObject. Insertion dans la table `gare` : `code_uic`, `nom_gare`, `latitude`, `longitude`, `iso_pays`.

1.4.3 Job_Operateur : Table cible : operateur (Job 3/9)

Objectif

Extraire les opérateurs ferroviaires depuis `02_bot_agencies.json` et les insérer dans la table `operateur`. La lecture JSON nécessite deux composants : `tFileInputRaw` + `tExtractJSONFields`.

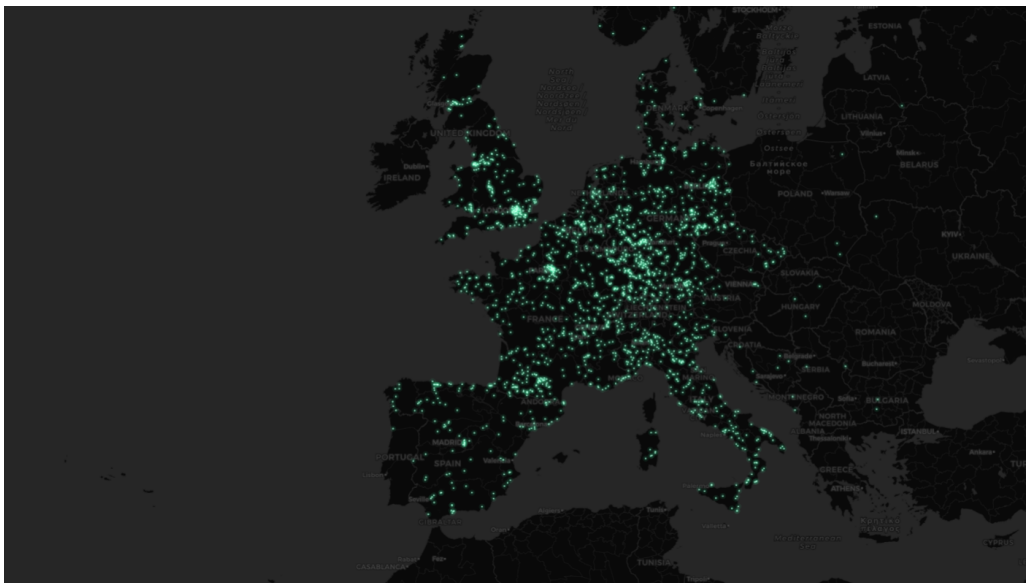


Figure 1.4: Schéma Talend : Job_Operateur

tFileInputRaw. Chargement du fichier JSON brut comme une seule chaîne de texte.

tExtractJSONFields. Extraction des champs : `agency_id` → `code_operateur`, `agency_name` → `nom_operateur`, `agency_country` → `iso_pays`. Résultat : 30 opérateurs extraits.

tDBObject. Insertion des 30 opérateurs dans la table `operateur`.

1.4.4 Job_TypeTrain : Table cible : type_train (Job 4/9)

Objectif

Extraire les types de trains uniques depuis `trips.json`. Le champ `trip_short_name` contient le code du service (ex. IR 11500) : on en extrait le préfixe alphabétique (IR) pour obtenir le type de train.

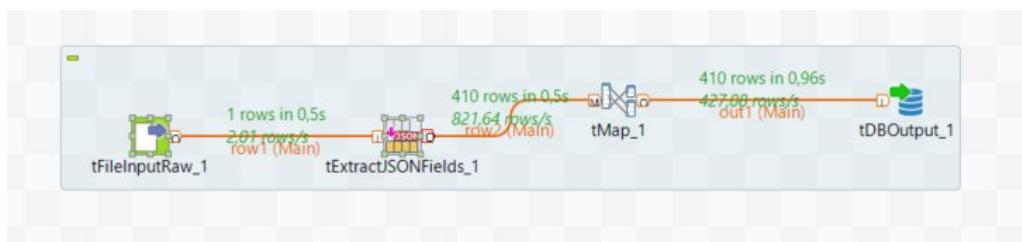


Figure 1.5: Schéma Talend : Job_TypeTrain

tFileInputRaw + tExtractJSONFields. 410 trajets extraits avant déduplication.

tMap + tUniqRow. Extraction du préfixe alphabétique, puis déduplication pour ne garder qu'une entrée par type (IR, EC, EN, ICE, TGV...). Le champ `id_type_train` est auto-incrémenté par PostgreSQL.

tDBOutput. Insertion dans la table `type_train` : colonne `type_nom`.

1.4.5 Job_Ligne : Table cible : ligne (Job 5/9)

Objectif

Extraire les 206 lignes ferroviaires depuis `routes.json` avec leur nom, distance et type de service (JOUR ou NUIT). Une ligne avec `route_id` null est détectée et filtrée (207 → 206).

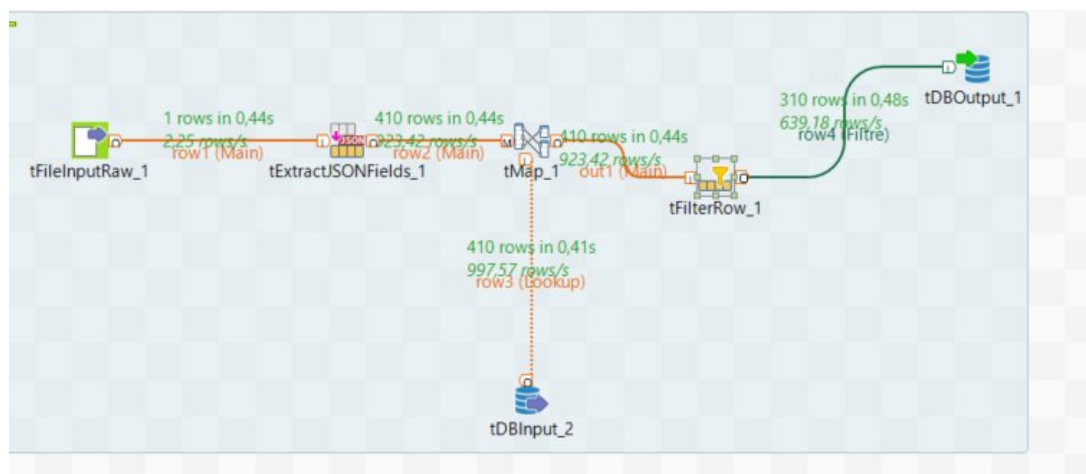


Figure 1.6: Schéma Talend : Job_Ligne

Composants. Lecture JSON → mapping des colonnes (`route_id` → `id_ligne`, `type_service` avec contrainte CHECK) → filtrage de la ligne invalide → insertion des 206 lignes dans la table `ligne`.

1.4.6 Job_Trajet : Table cible : `trajet` (Job 6/9)

Objectif

Extraire les 410 trajets depuis `trips.json` avec leurs gares de départ et d'arrivée, leurs horaires et leur ligne associée. C'est une table centrale : `itineraire` et `emission` en dépendent.

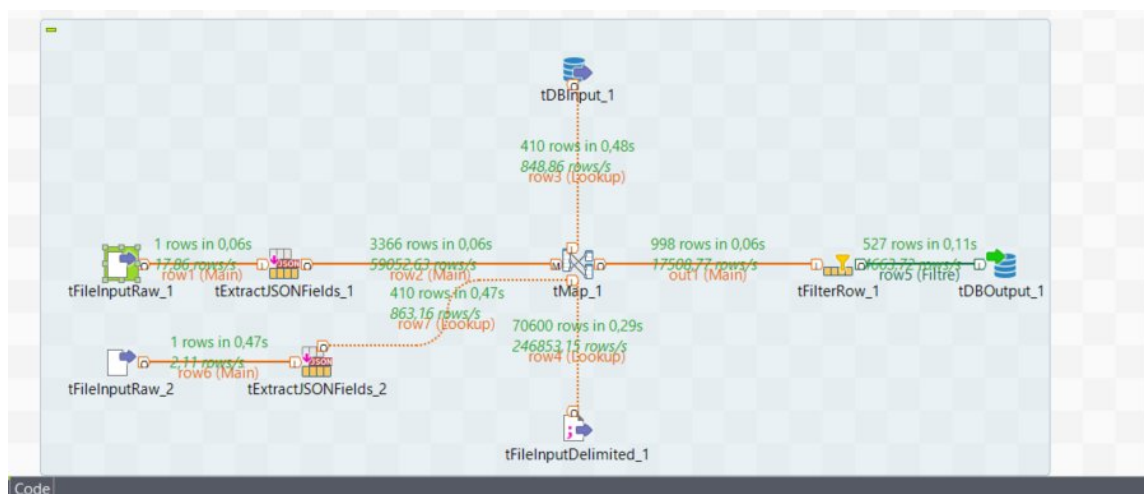


Figure 1.7: Schéma Talend : Job_Trajet

Composants. Extraction de `trip_id`, `trip_origin`, `trip_headsign`, horaires et `route_id`. Déduplication sur `trajet_id`. Les horaires sont conservés en `VARCHAR(30)` car Talend

ne convertit pas nativement les chaînes en type TIME PostgreSQL. Résultat : 410 trajets insérés.

1.4.7 Job_Exploite : Table cible : exploite (Job 7/9)

Objectif

Créer les associations entre opérateurs et lignes (table de liaison N-N). Certains `agency_id` sont composés (ex. CFR/MÁV/ÖBB) : on ne garde que le premier opérateur.

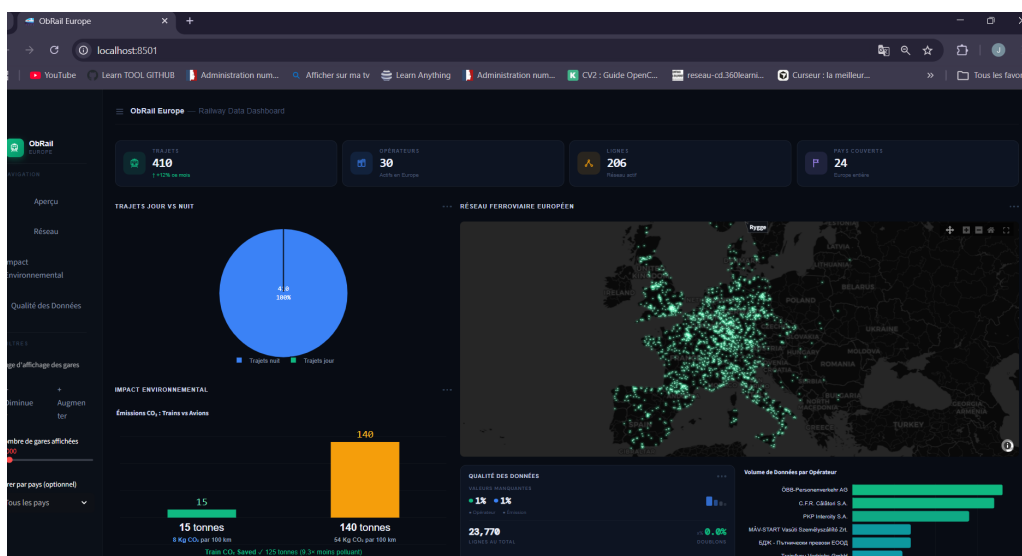


Figure 1.8: Schéma Talend : Job_Exploite

Composants. Extraction depuis `routes.json` → découpe des `agency_id` composés → filtrage de la ligne invalide → insertion des 206 associations. Clé primaire composite : (`code_operateur`, `id_ligne`).

1.4.8 Job_Itineraire : Table cible : itineraire (Job 8/9)

Objectif

C'est le job **le plus complexe** du pipeline. Il construit les itinéraires en associant chaque arrêt de `trip_stop.json` à son code UIC officiel (depuis `stations.csv`), à son trajet en base (via `tDBInput`), et aux villes intermédiaires (depuis `trips.json`). Trois *lookups* simultanés dans un seul `tMap`.

Sur 998 arrêts construits, **527 sont insérés**. Les 471 autres sont rejetés car leur nom de gare dans `trip_stop.json` ne correspond à aucun code UIC dans `stations.csv`.

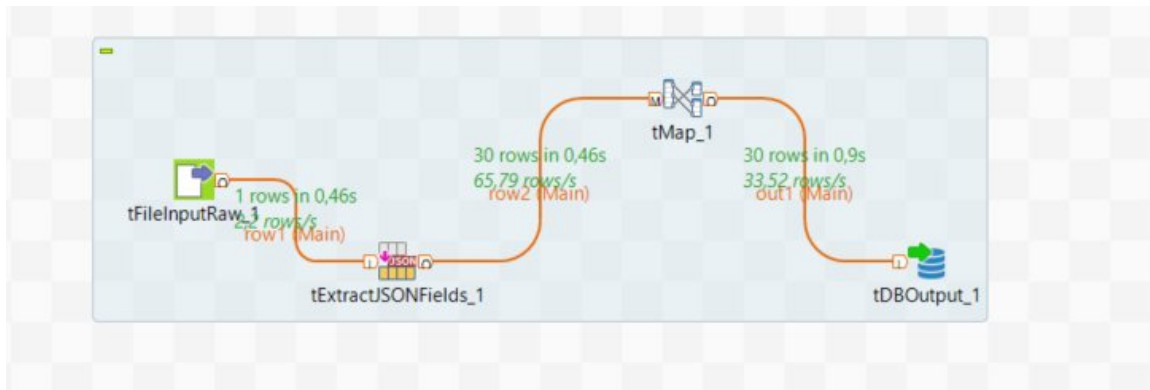


Figure 1.9: Schéma Talend : Job_Itineraire

tFileInputRaw_1 + tExtractJSONFields_1 : Source principale. Lecture de `trip_stop.json`. Champs : `trip_id`, `stop_sequence`, `stop_id`. 3 366 arrêts extraits.

tFileInputRaw_2 + tExtractJSONFields_2 : Lookup trips.json. Extraction du champ via (villes intermédiaires). Left Outer Join.

tDBInput : Lookup table trajet. Requête SQL : `SELECT trajet_id FROM trajet`. 410 trajets chargés en mémoire. Inner Join pour ne traiter que les arrêts dont le trajet existe en base.

tFileInputDelimited : Lookup stations.csv. Conversion du nom de gare en code UIC officiel. 70 600 lignes en *lookup*. Inner Join.

tMap : Triple jointure. Production de chaque ligne de la table *itineraire* : `trajet_id`, `chemin`, `code_uic`.

Résultat. 998 lignes issues du tMap → 527 insérées, 471 rejetées (UIC introuvable).

1.4.9 Job_Emission : Table cible : emission (Job 9/9)

Objectif

Extraire les données d'empreinte carbone depuis `trips.json` et insérer dans la table `emission`. L'empreinte train est lue directement (`emissions_co2e`). L'empreinte avion est calculée : $\text{distance} \times 0,158 \text{ kg CO}_2/\text{passager/km}$ (facteur ADEME/BEIS). Sur 410 trajets, **310 ont des données CO₂ renseignées**.



Figure 1.10: Schéma Talend : Job_Emission

Parsing. Les champs sont extraits en String car le JSON peut contenir la chaîne "null" à la place d'une vraie valeur nulle.

tMap : Calcul CO₂ avion. Jointure avec le *lookup* trajet. Conversion via `Double.parseDouble()` et calcul :

$$\text{empreinte_avion_kg} = \text{distance_km} \times 0,158 \quad (1.1)$$

tFilterRow. Rejet des 100 trajets sans données CO₂. Résultat : 310 émissions insérées.

1.5 Automatisation de l'intégration ETL

Une fois les 9 jobs Talend implémentés et validés, il est indispensable d'automatiser leur exécution. Sans automatisation, l'ingénieur doit relancer manuellement chaque job dès que les fichiers sources évoluent, ce qui est source d'oublis et d'incohérences.

1.5.1 Contexte et choix technique

Le pipeline ETL est exécuté sur un environnement Windows. Le mécanisme natif de planification est le **Planificateur de tâches Windows**, combiné à un script `.bat`. Cette solution ne nécessite aucune installation supplémentaire et s'intègre directement avec les scripts générés par TALEND OPEN STUDIO lors de l'export des jobs.

Lors de l'export, Talend génère pour chaque job un fichier `<nom_job>.run.bat` accompagné des fichiers Java compilés (`.jar`) et des dépendances nécessaires.

1.5.2 Structure du script de lancement

Le script principal `lancement.bat` constitue le point central d'orchestration du pipeline. Il s'organise en quatre parties :

Partie 1 : Configuration. Définition des variables centralisées :

- **BASE_DIR** : répertoire contenant les jobs Talend exportés.
- **LOG_DIR** : répertoire des fichiers de logs.
- **DUMP_DIR** : répertoire des sauvegardes PostgreSQL.

Le dossier de logs est créé automatiquement. Un fichier de log daté est généré à chaque exécution.

Partie 2 : Exécution séquentielle des 9 jobs. Chaque job est exécuté via une procédure interne `run_job` qui contrôle l'existence du fichier batch, lance l'exécution et vérifie le code de retour. Si le code est différent de zéro, le script interrompt immédiatement le pipeline pour éviter la propagation d'erreurs dans les tables dépendantes.

Partie 3 : Journalisation et traçabilité. Toutes les étapes sont enregistrées dans un fichier de log unique : début et fin de chaque job, succès ou échec, codes d'erreur, messages système. Les sorties standard et d'erreur sont redirigées vers ce fichier.

Partie 4 : Sécurité et intégrité. Le mot de passe PostgreSQL est saisi dynamiquement et supprimé en fin de processus. Avant l'exécution, un nettoyage de la base est effectué avec désactivation temporaire de `session_replication_role`. Une sauvegarde SQL complète est réalisée en fin de pipeline.

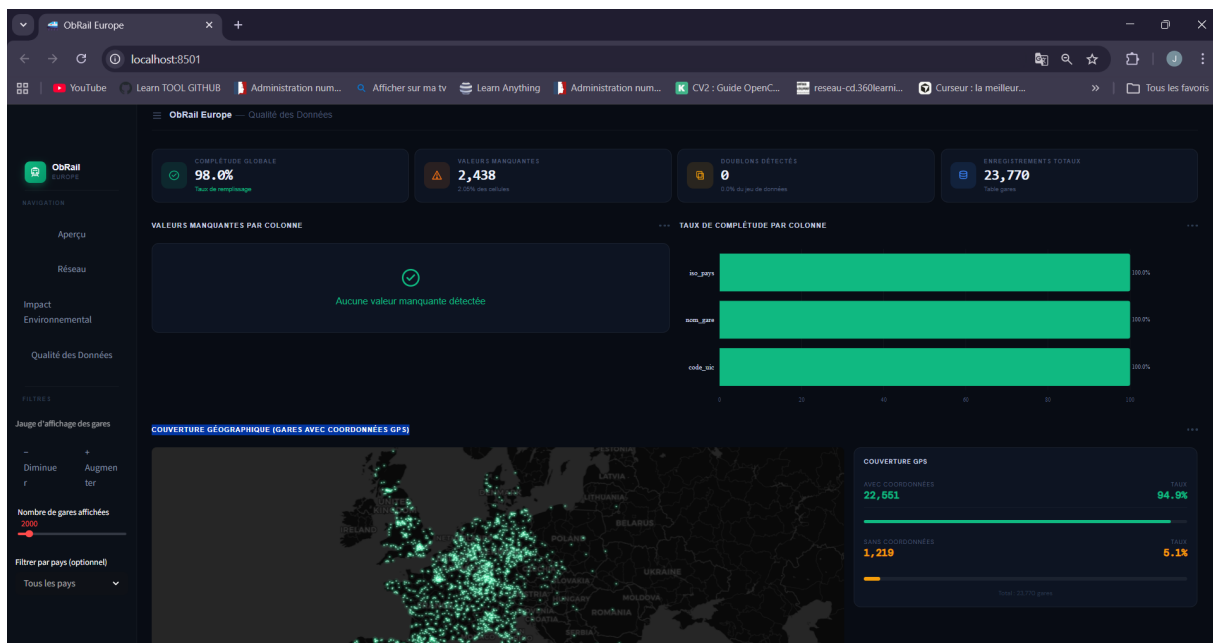


Figure 1.11: Processus d'automatisation du pipeline ETL

1.5.3 Équivalent sur Linux

Sur un environnement Linux, l'orchestration aurait été réalisée avec un script shell (`.sh`) et l'interpréteur Bash. La planification aurait pu être améliorée grâce à `cron`, permettant de déclencher les traitements de manière automatique et périodique.

CHAP 2 : L'API REST

Exposition des données via une interface web standardisée

L'API REST constitue la couche d'accès aux données du projet OBRail EUROPE. Elle permet à toute application cliente (dashboard, application mobile, service tiers) d'interroger la base de données de manière standardisée, sécurisée et performante.

2.1 Objectif du backend

Le backend expose une API REST permettant :

- La consultation des données ferroviaires (trajets, gares, lignes).
- La production de statistiques de pilotage (KPI).
- L'alimentation du dashboard Streamlit.

Le service est conçu en **architecture modulaire** : routes → services → models → database.

2.2 Architecture technique

2.2.1 Structure des dossiers

```
1 backend/
2   app/
3     main.py           # Instanciation FastAPI + routers
4     database.py       # Engine SQLAlchemy + sessions
5     routes/          # Endpoints HTTP
6     services/        # Logique métier / acces base
7     models/          # Mapping ORM des tables PostgreSQL
8     schemas/         # Schemas de reponse Pydantic
9     requirements.txt
```

Listing 2.1: Arborescence du backend

2.2.2 Rôle de chaque couche

Table 2.1: Rôle de chaque module du backend

Module	Rôle
<code>main.py</code>	Instancie l'application FastAPI et enregistre les routers.
<code>database.py</code>	Crée l'engine SQLAlchemy, la session DB et Base.
<code>models/</code>	Mapping ORM des tables PostgreSQL.
<code>schemas/</code>	Schémas de réponse Pydantic (validation et sérialisation).
<code>services/</code>	Logique métier et accès à la base de données.
<code>routes/</code>	Définition des endpoints HTTP.

2.3 Prérequis et lancement

2.3.1 Dépendances Python

Les dépendances sont déclarées dans `backend/requirements.txt` : `fastapi`, `uvicorn`, `sqlalchemy`, `psycopg2-binary`, `pydantic`, `python-dotenv`, `alembic`, `pytest`, `httpx`.

2.3.2 Configuration et lancement local

La variable d'environnement `DATABASE_URL` doit être définie dans un fichier `.env` :

```
1 DATABASE_URL=postgresql://postgres:<mot_de_passe>@localhost:5432/obrail
```

Listing 2.2: Configuration de la connexion

```
1 cd backend
2 source ../venv/bin/activate
3 uvicorn app.main:app --reload --port 8000
```

Listing 2.3: Lancement du serveur

2.3.3 URLs utiles

URL	Description
<code>http://127.0.0.1:8000</code>	API de base
<code>http://127.0.0.1:8000/docs</code>	Interface Swagger UI (documentation interactive)
<code>http://127.0.0.1:8000/redoc</code>	Documentation ReDoc
<code>http://127.0.0.1:8000/openapi.json</code>	Spécification OpenAPI

2.4 Configuration FastAPI

Le fichier `app/main.py` enregistre les routers suivants : `health_routes`, `trajet_routes`, `gare_routes`, `ligne_routes`, `stats_routes`. Tous les endpoints sont exposés dans un seul service FastAPI.

2.5 Connexion base de données

Le fichier `app/database.py` utilise `create_engine(DATABASE_URL)` pour la connexion PostgreSQL. La fonction `get_db()` sert de *dependency* FastAPI pour injecter une session DB proprement, avec fermeture automatique dans le bloc `finally`.

2.6 Modèles de données principaux (ORM)

Les modèles utilisés activement par les routes sont : `Trajet`, `Gare`, `Ligne`, `Operateur`, `Emission`, `Itineraire`.

Relations majeures :

- `Trajet.id_ligne` → `Ligne.id_ligne`
- `Itineraire.trajet_id` → `Trajet.trajet_id`
- `Itineraire.code_uic` → `Gare.code_uic`

2.7 Schémas de réponse (Pydantic)

```
1 # TrajetResponse
2 {"trajet_id": "string", "id_ligne": 0,
3  "gare_depart": "string", "gare_arrivee": "string",
4  "heure_depart": "string", "heure_arrivee": "string"}
5
6 # GareResponse
7 {"code_uic": "string", "nom_gare": "string",
8  "longitude": 0.0, "latitude": 0.0, "iso_pays": "string"}
9
10 # LigneResponse
11 {"id_ligne": 0, "nom_ligne": "string",
12  "distance": 0.0, "type_service": "JOUR|NUIT|null"}
```

Listing 2.4: Schémas de réponse Pydantic

2.8 Documentation complète des endpoints

Table 2.2: Endpoints de l'API REST OBRail EUROPE

Catégorie	Méthode	Endpoint	Description
Santé	GET	/health	Contrôle de disponibilité
Trajets	GET	/trajets/	Liste complète des trajets
	GET	/trajets/{trajet_id}	Trajet par identifiant (404 si inexistant)
Gares	GET	/gares/	Liste de toutes les gares
Lignes	GET	/lignes/	Liste de toutes les lignes
Statistiques	GET	/stats/trajets/count	Nombre total de trajets
	GET	/stats/lignes/count	Nombre total de lignes
	GET	/stats/gares/count	Nombre total de gares
	GET	/stats/trajets/type	Répartition par type de service
	GET	/stats/emissions	Moyenne des empreintes CO ₂
	GET	/stats/operateurs	Volumes par opérateur
	GET	/stats/trajets/map	Segments géographiques pour carte

2.9 Gestion des erreurs et codes HTTP

Table 2.3: Codes HTTP utilisés

Code	Signification
200 OK	Succès de la requête.
307 Temporary Redirect	Appel sans slash terminal (redirection automatique FastAPI).
404 Not Found	Trajet inexistant.
500 Internal Server Error	Erreur serveur ou base de données.

Bonne pratique

Toujours utiliser les URLs avec slash final pour les endpoints de liste. Monitorer les logs Uvicorn en environnement de développement.

2.10 Contrat backend ↔ dashboard

Le dashboard dépend explicitement des formats JSON des endpoints `/stats/*`, des schémas `GareResponse`, `TrajetResponse` et `LigneResponse`. Tout changement de nom de champ dans l'API doit être répercuté dans : `dashboard/services/api_service.py`, `dashboard/app.py`, `dashboard/components/*.py`.

2.11 Vérification rapide (*smoke tests*)

```
1 curl http://127.0.0.1:8000/health
2 curl http://127.0.0.1:8000/stats/trajets/count
3 curl http://127.0.0.1:8000/stats/trajets/type
4 curl http://127.0.0.1:8000/stats/emissions
5 curl http://127.0.0.1:8000/stats/trajets/map
```

Listing 2.5: Tests de vérification après démarrage

Résultat attendu : pas d'erreur 500, JSON valide sur chaque endpoint.

2.12 Évolutions recommandées

1. Ajouter une couche CRUD plus propre avec injection DB uniforme.
2. Ajouter des schémas de réponse explicites pour les endpoints `/stats/*`.
3. Ajouter une pagination (`limit/offset`) sur `/trajets/` et `/gares/`.
4. Ajouter des filtres par paramètres (`type_service`, `pays`, `opérateur`).
5. Ajouter des tests API automatisés (`pytest` + `httpx`) endpoint par endpoint.
6. Ajouter une gestion centralisée des erreurs (handlers FastAPI).

CHAP 3 : LE DASHBOARD

Visualisation interactive des données ferroviaires européennes

Le dashboard OBRail EUROPE a pour objectif de visualiser, analyser et valoriser les données ferroviaires européennes à travers une interface interactive et moderne. Il représente la couche de restitution du projet, permettant aux utilisateurs finaux d'exploiter concrètement le travail réalisé dans les parties précédentes.

3.1 Fonctionnalités principales du dashboard

3.1.1 Aperçu global

Cette page présente les indicateurs clés du projet sous forme de cartes synthétiques : nombre total de trajets, de gares, de lignes, d'opérateurs et de pays européens couverts. Ces métriques donnent immédiatement une idée de l'ampleur du réseau documenté.



Figure 3.1: Dashboard : Aperçu global des indicateurs clés

3.1.2 Cartographie du réseau

Une carte interactive de l'Europe montre l'ensemble des gares et des segments ferroviaires. Chaque gare est représentée par un point, et les liaisons sont matérialisées par des segments colorés. L'utilisateur peut zoomer, se déplacer et filtrer les données par pays pour se concentrer sur une zone géographique précise.

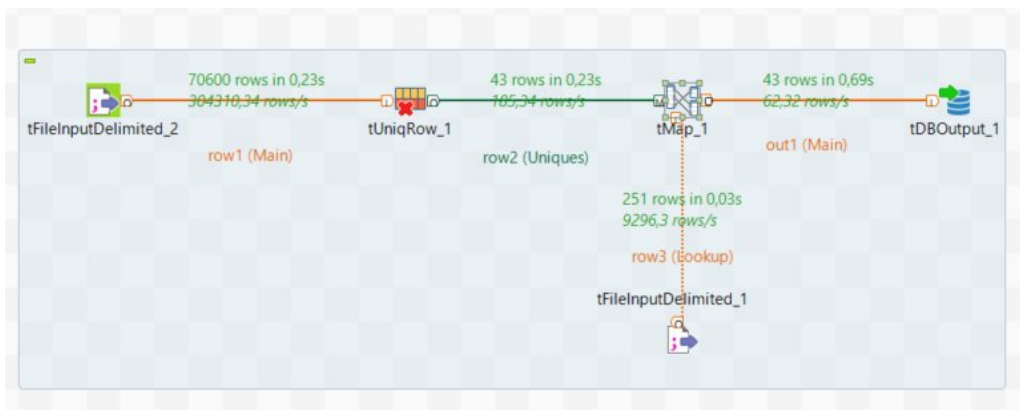
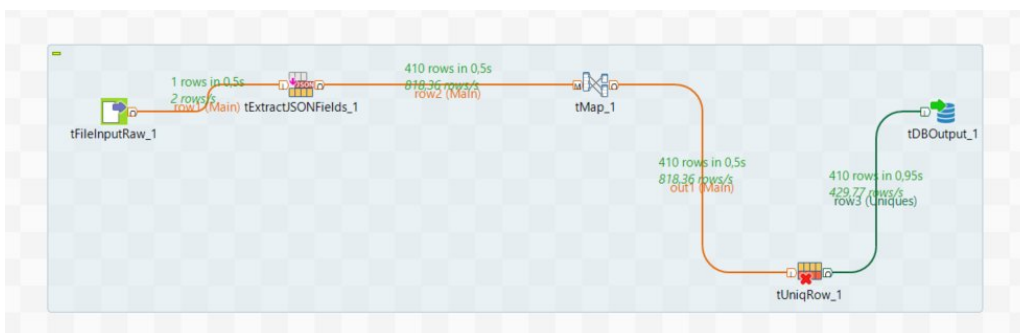


Figure 3.2: Dashboard : Cartographie interactive du réseau ferroviaire européen

3.1.3 Impact environnemental

Cette page compare les émissions de CO₂ du train et de l'avion. Elle affiche les facteurs d'émission moyens (8 kg CO₂/100 km pour le train contre 54 kg CO₂/100 km pour l'avion), calcule les économies totales générées par les trajets en train, et présente un classement des opérateurs en fonction de leur contribution à la réduction des émissions.

Figure 3.3: Dashboard : Comparaison des émissions CO₂ train vs avion

3.1.4 Qualité des données

Cette page fournit un diagnostic complet de la base : taux de complétude global, colonnes contenant des valeurs manquantes, doublons détectés et couverture GPS des gares. Ces informations sont essentielles pour les équipes techniques qui doivent maintenir et améliorer continuellement la qualité du référentiel.

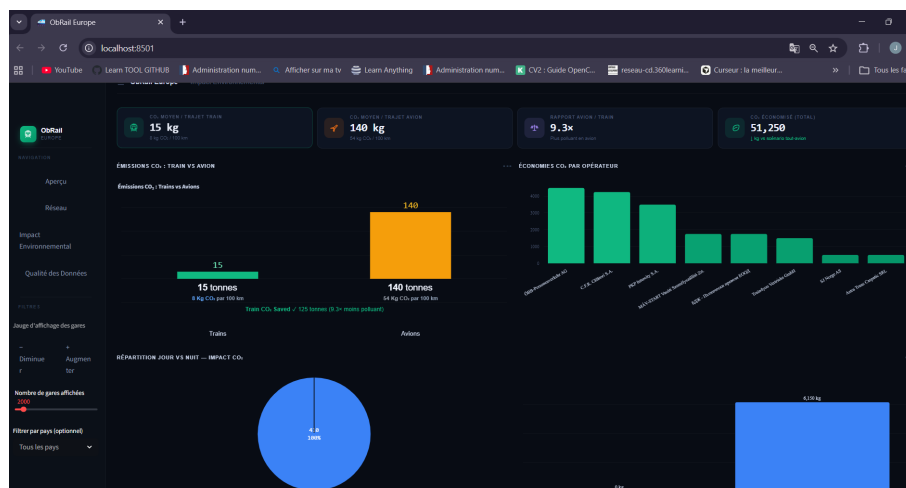


Figure 3.4: Dashboard : Diagnostic de qualité des données

3.2 Technologies utilisées et justification des choix

Table 3.1: Technologies du dashboard et justification

Technologie	Justification
Streamlit	Framework Python permettant de transformer un script d'analyse en application web interactive sans maîtriser le front-end. Choix pragmatique pour une équipe IA : Python pur, interface générée automatiquement.
Plotly	Graphiques interactifs dans le navigateur avec zoom, survol et filtrage. Intégration native avec Streamlit, couvrant graphiques classiques et cartes géographiques.
Plotly Mapbox + CartoDB	Rendu cartographique fluide même avec des milliers de points. Le style <i>Dark Matter</i> s'intègre harmonieusement avec le thème du dashboard.
Requests + Pandas	Requests pour les appels API centralisés dans un service dédié. Pandas pour le filtrage, le regroupement et la préparation des données.
Cache Streamlit	Décorateur <code>@st.cache_data</code> pour éviter les réexecutions coûteuses. Amélioration drastique des performances.
HTML/CSS personnalisé	Stylisation des cartes d'indicateurs avec icônes SVG, charte graphique cohérente (polices DM Sans et DM Mono), animations au survol.

3.3 Apports du dashboard au projet OBRail EUROPE

Exploitation des données. Le dashboard permet à n'importe qui de consulter instantanément les métriques clés. Les informations, autrefois enfouies dans des tables SQL, sont maintenant visibles en un clic et mises à jour automatiquement.

Arguments environnementaux. Les utilisateurs peuvent voir immédiatement que le train émet en moyenne 8 kg de CO₂ pour 100 km contre 54 kg pour l'avion, et quantifier les économies totales. Ces chiffres sont essentiels pour les campagnes de communication d'OBRail EUROPE et de ses partenaires.

Contrôle qualité. Les équipes techniques peuvent détecter rapidement les anomalies, baisses de complétude ou doublons. Cette transparence est indispensable pour maintenir la crédibilité de l'observatoire.

Navigation intuitive. Menu latéral, graphiques interactifs avec zoom et filtrage, boutons d'ajustement de la densité d'affichage.

Performances optimisées. Cache pour éviter les appels API redondants, échantillonnage limité à 1 000 segments et 2 000 gares pour la carte.

CONCLUSION

Ce projet MSPR a permis de mettre en place un écosystème complet de gestion et d'analyse des données ferroviaires européennes pour OBRail EUROPE, de la collecte brute jusqu'à la visualisation interactive.

Partie 1 : Les données. Nous avons conçu et implémenté un pipeline ETL robuste avec TALEND OPEN STUDIO. Nous avons sélectionné et justifié neuf sources de données ferroviaires, conçu un modèle conceptuel de données cohérent avec neuf entités principales, et développé neuf jobs ETL automatisés pour alimenter une base PostgreSQL. Nous avons également assuré la conformité RGPD du traitement.

Partie 2 : L'API REST. Nous avons exposé ces données via une API développée avec FastAPI, proposant des endpoints de consultation et des endpoints statistiques spécialement conçus pour le dashboard.

Partie 3 : Le Dashboard. Nous avons créé un tableau de bord interactif avec Streamlit et Plotly permettant de visualiser le réseau ferroviaire, de comparer l'impact environnemental train/avion, de surveiller la qualité des données et d'accéder aux indicateurs clés.

Apports du projet

Ce projet démontre qu'un pipeline ETL bien conçu, une API REST robuste et un dashboard ergonomique forment un écosystème complet et cohérent, capable de transformer des données dispersées en *insights* actionnables.

Pour OBRail EUROPE, c'est désormais une infrastructure opérationnelle qui permet de collecter automatiquement des données, de les structurer dans un référentiel unique, et de les rendre accessibles à différents profils d'utilisateurs. Au-delà de l'aspect technique, ce projet contribue directement à la mission d'OBRail EUROPE : promouvoir le train comme alternative écologique à l'avion en s'appuyant sur des données objectives et vérifiables.

Perspectives d'amélioration

1. **Enrichissement des données.** Intégration de nouvelles sources : horaires en temps réel, données de fréquentation, informations tarifaires.

2. **Analyse temporelle.** Ajout d'un filtre temporel pour analyser l'évolution du réseau : croissance des liaisons, tendances des émissions par pays ou par opérateur.
3. **Modèles prédictifs.** Développement de modèles d'intelligence artificielle : prédiction de la demande, optimisation des itinéraires, recommandation de trajets selon des critères environnementaux.
4. **Export et partage.** Fonctionnalités d'export des graphiques en PNG/PDF et génération automatique de rapports périodiques.
5. **Tests automatisés.** Mise en place de tests unitaires et d'intégration pour garantir la stabilité lors des évolutions futures.

Ce projet illustre la complémentarité entre ingénierie des données, développement d'API et visualisation interactive. Sans données propres, l'API n'a rien à exposer ; sans API, le dashboard ne peut rien afficher ; sans dashboard, les données restent inaccessibles. C'est exactement ce dont OBRail EUROPE avait besoin : un système complet, automatisé, évolutif et accessible, qui transforme la complexité technique en valeur métier concrète.

BIBLIOGRAPHIE ET WEBOGRAPHIE

- [1] **transport.data.gouv.fr** : Plateforme nationale des données de transport. Fichier `stations.csv` : liste des gares européennes.
<https://transport.data.gouv.fr>
- [2] **Back-on-Track** : ONG européenne pour les trains de nuit. Données ferroviaires structurées (agencies, routes, trips, trip_stop).
<https://back-on-track.eu> | Dépôt GitHub associé.
- [3] **ISO 3166** : Standard international des codes pays.
<https://www.iso.org/iso-3166-country-codes.html>
- [4] **ADEME** : Agence de l'Environnement et de la Maîtrise de l'Énergie. Facteurs d'émission CO₂ des transports.
<https://www.ademe.fr>
- [5] **BEIS** : UK Department for Business, Energy & Industrial Strategy. Facteur d'émission avion : 0,158 kg CO₂/passager/km.
- [6] **Eurostat** : Office statistique de l'Union européenne. Statistiques ferroviaires.
<https://ec.europa.eu/eurostat>
- [7] **Commission européenne** : Green Deal européen et programme TEN-T.
https://ec.europa.eu/info/strategy/priorities-2019-2024/european-green-deal_fr
- [8] **PostgreSQL** : Système de gestion de base de données relationnelle open source.
<https://www.postgresql.org>
- [9] **Talend Open Studio** : Outil ETL open source pour l'intégration de données.
<https://www.talend.com/products/talend-open-studio/>
- [10] **FastAPI** : Framework web Python moderne pour la création d'API.
<https://fastapi.tiangolo.com>
- [11] **Streamlit** : Framework Python pour la création d'applications web de données.
<https://streamlit.io>
- [12] **Plotly** : Bibliothèque de visualisation interactive.
<https://plotly.com/python/>
- [13] **SQLAlchemy** : Toolkit SQL et ORM Python.
<https://www.sqlalchemy.org>
- [14] **Pydantic** : Validation de données et gestion des schémas en Python.
<https://pydantic-docs.helpmanual.io>